

Entrega 1 — A base do Sistema de Registro Acadêmico

Modalidade	Início	Entrega	Peso
Grupos de 1 a 3 alunos	Em aula	Próxima aula, até o início do encontro	1ª de 4 entregas do Trabalho da Disciplina

1. Por que esta entrega existe

O Trabalho da Disciplina é um Sistema de Registro Acadêmico completo: cadastro de alunos e disciplinas, lançamento de notas e interface gráfica. Ele não será construído de uma vez. Vamos construí-lo em quatro entregas incrementais — que é exatamente como o RAD funciona.

Entrega	O que você constrói	Quando
1 — Base	Modelo de dados + banco + tabelas + CRUD em modo texto	esta entrega
2 — Consultas	Relatórios com SELECT, WHERE e JOIN	após a aula de recuperação de registros
3 — Interface	As mesmas funções, agora com telas Tkinter	Tema 4
4 — Aplicação final	Sistema integrado, apresentado em aula	APS

Se a base ficar bem feita agora, as entregas seguintes só acrescentam camadas. Se a base ficar torta, você vai reescrever tudo três vezes. **Esse é o ponto pedagógico da atividade.**

2. O cenário

Uma instituição de ensino de pequeno porte controla alunos, disciplinas e notas em planilhas espalhadas entre a secretaria e os professores. Há matrículas duplicadas, nomes de disciplinas escritos de três formas diferentes e notas que somem. A coordenação quer um sistema simples que centralize esses dados com integridade garantida pelo banco.

Nesta entrega você não precisa resolver o problema inteiro. Precisa construir **a fundação sobre a qual ele será resolvido.**

3. Modelo de dados exigido

Três tabelas. Use exatamente estes nomes de tabela e de coluna — isso permite que o script de autoteste funcione e agiliza a correção.

Tabela aluno

Coluna	Tipo	Restrições
id	INTEGER	PRIMARY KEY AUTOINCREMENT
matricula	TEXT	NOT NULL, UNIQUE
nome	TEXT	NOT NULL
email	TEXT	—
data_nascimento	TEXT	formato AAAA-MM-DD
ativo	INTEGER	NOT NULL, DEFAULT 1 (1 = sim, 0 = não)

Tabela disciplina

Coluna	Tipo	Restrições
id	INTEGER	PRIMARY KEY AUTOINCREMENT
codigo	TEXT	NOT NULL, UNIQUE
nome	TEXT	NOT NULL
carga_horaria	INTEGER	NOT NULL
periodo	INTEGER	—

Tabela inscricao — a tabela que resolve o relacionamento

Um aluno cursa várias disciplinas; uma disciplina tem vários alunos. Isso é um relacionamento **N:N**, e o SQL não sabe representar N:N com duas tabelas só. Precisamos de uma terceira, chamada **tabela associativa**.

Coluna	Tipo	Restrições
id	INTEGER	PRIMARY KEY AUTOINCREMENT
aluno_id	INTEGER	NOT NULL, FOREIGN KEY → aluno(id)
disciplina_id	INTEGER	NOT NULL, FOREIGN KEY → disciplina(id)
nota1	REAL	—
nota2	REAL	—
—	—	UNIQUE (aluno_id, disciplina_id)

Pergunta que você deve saber responder na correção: por que nota1 fica em inscricao e não em aluno nem em disciplina?

4. O que entregar

Um arquivo .zip com o nome ARA0095_Entrega1_<sobrenomes>.zip contendo:

projeto/	
— database.py	conexão + criação das tabelas
— crud_aluno.py	5 funções de CRUD de aluno
— crud_disciplina.py	5 funções de CRUD de disciplina
— main.py	menu de texto
— seed.py	carga de dados de exemplo
— autoteste.py	(não modifique)
— README.md	preenchido
— modelo_dados.png	imagem do modelo (ou .pdf)

Não inclua o arquivo escola.db nem a pasta __pycache__. O banco é gerado pelo código; se o código não recria o banco do zero, ele está errado.

O modelo de dados (modelo_dados.png)

Um diagrama simples mostrando as três tabelas, as chaves primárias, as chaves estrangeiras e a cardinalidade. Pode ser feito em qualquer ferramenta (draw.io, dbdiagram.io, Lucidchart) ou até desenhado à mão e fotografado, desde que legível.

5. Requisitos técnicos (não negociáveis)

1. SGBD: SQLite, via módulo sqlite3 da biblioteca padrão. Sem instalar nada.
2. Parâmetros com ?. Todo valor vindo do usuário entra no SQL como ? e vai em uma tupla no execute. Montar SQL com f-string ou com + zera o item 4 da rubrica — é a brecha de SQL Injection que discutimos em aula.
3. commit() depois de todo INSERT, UPDATE e DELETE. Sem ele, o banco descarta a alteração ao fechar a conexão.
4. WHERE obrigatório em UPDATE e DELETE. Um DELETE FROM aluno sem WHERE apaga a instituição inteira.
5. Separação de responsabilidades. Nenhum input() ou print() de menu dentro dos arquivos crud_*.py. A conversa com o usuário mora só em main.py. É isso que vai permitir, na Entrega 3, trocar o menu por telas Tkinter sem reescrever o CRUD.
6. Tratamento de erro no cadastro duplicado. Inserir uma matrícula que já existe deve exibir uma mensagem clara ao usuário, não um traceback.
7. PRAGMA foreign_keys = ON em toda conexão. Por padrão o SQLite ignora chaves estrangeiras.

6. Roteiro sugerido

Em aula (com o professor por perto — aproveite)

1. Baixe o código base e rode python autoteste.py. Ele vai falhar em quase tudo. Normal: é o seu mapa.
2. Desenhe o modelo de dados no papel, em grupo, antes de escrever qualquer linha.
3. Implemente os TODOs 1 e 2 (tabelas disciplina e inscricao) e rode python database.py.
4. Abra o escola.db no DB Browser for SQLite e confira visualmente se as tabelas nasceram certas.

5. Implemente os TODOs 3, 4 e 5 (o CRUD de aluno). Rode o autoteste de novo: a seção 2 deve ficar verde.

Dúvida travando o grupo? Chame o professor agora, não no domingo.

Fora de aula

1. TODO 6: o CRUD de disciplina, espelhando o de aluno.
2. TODOs 7, 8 e 9: completar os menus.
3. Rode python seed.py e depois python main.py, e teste o sistema como um usuário faria.
4. Rode python autoteste.py até dar 21 OK / 0 FALHA.
5. Preencha o README.md e exporte o modelo_dados.png.
6. Feche o .zip e envie.

7. Como você será avaliado

#	Critério	Pontos
1	Modelo de dados — três tabelas, tipos e restrições corretos, FKs declaradas, UNIQUE composto em inscricao, diagrama legível e coerente com o código	2,0
2	CRUD de aluno — as 5 funções funcionando, com retorno correto (id, None, True, False)	2,0
3	CRUD de disciplina — as 5 funções funcionando	2,0
4	Qualidade técnica — parâmetros ?, commit, close, WHERE, separação entre menu e CRUD, tratamento de duplicidade	2,0
5	Menu, README e organização — sistema navegável sem travar, README preenchido com o modelo e as decisões, nomes de arquivo conforme o padrão	2,0
	Total	10,0

Descontos

- escola.db ou __pycache__ no zip: -0,5
- Código que não roda por erro de sintaxe: -2,0
- SQL montado com f-string: item 4 zerado

8. Sobre uso de IA

Você pode usar assistentes de IA. Mas o grupo inteiro será **arguido oralmente** sobre o código na aula de entrega: qualquer integrante pode ser chamado a explicar qualquer trecho, ou a fazer uma alteração pequena ao vivo. **Código que o grupo não sabe explicar vale zero**, independentemente de funcionar. Registre no README o que foi gerado com auxílio de IA e o que o grupo revisou — isso é boa prática profissional, não delação.

9. Erros que derrubam a maioria dos grupos

Sintoma	Causa quase sempre
no such table: disciplina	Rodou main.py antes de database.py, ou o TODO 1 ficou pendente
Alterou o registro mas voltou ao valor antigo	Faltou conexao.commit()

Sintoma	Causa quase sempre
UNIQUE constraint failed	Está tentando cadastrar matrícula ou código que já existe — isso é o banco protegendo você
table aluno already exists	Faltou o IF NOT EXISTS no CREATE TABLE
Excluiu um aluno e as notas dele viraram órfãs	Faltou PRAGMA foreign_keys = ON
cursor.rowcount sempre 0	O WHERE não casou com nenhum registro — confira o id
Menu trava com ValueError	Usuário digitou letra onde o código esperava número; use a função ler_inteiro()

10. Material de apoio

- Conteúdo digital, Tema "Python com banco de dados" — módulos 2 e 3
- DB Browser for SQLite (<https://sqlitebrowser.org>) — para inspecionar o .db visualmente. Instale: vai poupar horas de depuração.
- Documentação do sqlite3: <https://docs.python.org/pt-br/3/library/sqlite3.html>
- Vídeo: Python SQLite Tutorial: Complete Overview — <https://www.youtube.com/watch?v=pd-0G0MigUA>

Uma última coisa. O objetivo aqui não é o CRUD. É você terminar a semana com um projeto que roda, tem estrutura e aceita crescer — porque nas próximas entregas ele vai crescer bastante. Faça a base direito.